Object Oriented Programming (CSEG1044)

Unit 03 – Lecture 02: Exception Handling (Core)

April 1, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O
- **Course thread:** Use Exception Handling (Core) to make Campus Resource Manager safer, more modular, and easier to recover when something goes wrong.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Recall Before You Start

- **Control flow**: try, catch, and finally are still flow-control ideas for abnormal situations. Main reference: `Unit 01 / Lecture 04 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Recall Before You Start

- **Control flow**: try, catch, and finally are still flow-control ideas for abnormal situations. Main reference: `Unit 01 / Lecture 04 slides/notes`.
- **Validation design**: Encapsulated objects should reject invalid state changes in a disciplined way. Main reference: `Unit 02 / Lecture 01 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Distinguish exception vs error; checked vs unchecked.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Distinguish exception vs error; checked vs unchecked.
- Write `try/catch/finally` and use `throw` vs `throws`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Learning Outcomes

- Distinguish exception vs error; checked vs unchecked.
- Write `try/catch/finally` and use `throw` vs `throws`.
- Design a small custom exception for validation.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Terms to Know

- **Exception**: abnormal condition a program can potentially handle.
- **Error**: serious problem typically not recoverable (e.g., `OutOfMemoryError`).
- **Checked exception**: must be caught or declared (e.g., `IOException`).
- **Unchecked exception**: subclass of `RuntimeException` (e.g., `NullPointerException`).
- **Stack trace**: call history showing where an exception occurred.
- **Handler**: catch block that responds to an exception.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Exceptions vs Errors* → *What happens when an exception occurs? (stack unwinding)* → *How to read a stack trace (very important skill)* → *Checked vs unchecked exceptions (when to use which?)*.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Exceptions vs Errors* → *What happens when an exception occurs? (stack unwinding)* → *How to read a stack trace (very important skill)* → *Checked vs unchecked exceptions (when to use which?)*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Exceptions vs Errors* → *What happens when an exception occurs? (stack unwinding)* → *How to read a stack trace (very important skill)* → *Checked vs unchecked exceptions (when to use which?)*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Unit 03: Nested Classes, Exceptions, Multithreading & IO Streams

- Lecture focus: Exception Handling (Core)

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Exceptions vs errors; checked vs unchecked

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Exceptions vs errors; checked vs unchecked
- ‘try/catch/finally‘, multiple catch; ‘throw‘ vs ‘throws‘

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Today: Roadmap

- Exceptions vs errors; checked vs unchecked
- ‘try/catch/finally‘, multiple catch; ‘throw‘ vs ‘throws‘
- Exception handlers + basic custom exception design for validation

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Exception vs Error

- Exceptions: conditions you can handle (invalid input, missing file).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Exception vs Error

- Exceptions: conditions you can handle (invalid input, missing file).
- Errors: serious JVM/system problems (usually not recoverable).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checked vs Unchecked

- Checked: must be caught or declared (`throws`) to compile.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checked vs Unchecked

- Checked: must be caught or declared (`throws`) to compile.
- Unchecked: `RuntimeException` (compiler does not force handling).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

`try/catch/finally` (pattern)

- Put risky code in `try`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

`try/catch/finally` (pattern)

- Put risky code in `try`.
- Catch specific exceptions first.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

`try/catch/finally` (pattern)

- Put risky code in `try`.
- Catch specific exceptions first.
- Use `finally` for cleanup (runs even if exception occurs).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

throw vs throws

- `throw`: create and throw an exception now.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

throw vs throws

- `throw`: create and throw an exception now.
- `throws`: declare that a method may throw an exception.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: throw (happens now)

```
static int divide(int a, int b) {
    if (b == 0) {
        throw new IllegalArgumentException("b must not be 0
            ");
    }
    return a / b;
}
```

- Thrown at runtime when `b == 0`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: throw (happens now)

```
static int divide(int a, int b) {
    if (b == 0) {
        throw new IllegalArgumentException("b must not be 0
            ");
    }
    return a / b;
}
```

- Thrown at runtime when `b == 0`.
- Execution jumps to a matching `catch`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: throws (declared in signature)

```
static void pauseOneSecond() throws InterruptedException {
    Thread.sleep(1000); // checked exception
}
```

- `throws` tells the caller: catch or declare.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: throws (declared in signature)

```
static void pauseOneSecond() throws InterruptedException {
    Thread.sleep(1000); // checked exception
}
```

- `throws` tells the caller: catch or declare.
- Checked exceptions are enforced by the compiler.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Caller must handle (try/catch) or declare (throws)

```
public static void main(String[] args) {
    try {
        pauseOneSecond();
    } catch (InterruptedException ex) {
        System.out.println("Interrupted");
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *Exception Handling (Core)* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Exceptions vs Errors*, *What happens when an exception occurs? (stack unwinding)*, the worked example, or the recap?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Full example (1/2): try block

```
import java.util.Scanner;

public class DivideDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter a: ");
            int a = Integer.parseInt(sc.nextLine());
            System.out.print("Enter b: ");
            int b = Integer.parseInt(sc.nextLine());

            int q = a / b;
            System.out.println("a/b = " + q);
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Full example (2/2): catch + finally

```
        } catch (NumberFormatException ex) {
            System.out.println("Please enter valid integers
                .");
        } catch (ArithmeticException ex) {
            System.out.println("Cannot divide by zero.");
        } finally {
            sc.close();
            System.out.println("Cleanup done (finally).");
        }
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

How the example behaves

- Valid input: prints quotient, then runs `finally`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

How the example behaves

- Valid input: prints quotient, then runs `finally`.
- Invalid integer input: `NumberFormatException` catch runs, then `finally`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

How the example behaves

- Valid input: prints quotient, then runs `finally`.
- Invalid integer input: `NumberFormatException` catch runs, then `finally`.
- Divide by zero: `ArithmeticException` catch runs, then `finally`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Execution sequence (try/catch/finally)

1. Enter `try` and execute statements top-to-bottom.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Execution sequence (try/catch/finally)

1. Enter `try` and execute statements top-to-bottom.
2. If no exception: skip all `catch` blocks.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Execution sequence (try/catch/finally)

1. Enter `try` and execute statements top-to-bottom.
2. If no exception: skip all `catch` blocks.
3. If an exception occurs: jump to the **first matching** `catch`.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Execution sequence (try/catch/finally)

1. Enter `try` and execute statements top-to-bottom.
2. If no exception: skip all `catch` blocks.
3. If an exception occurs: jump to the **first matching** `catch`.
4. After `try` or `catch`: always execute `finally` (cleanup).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Execution sequence (try/catch/finally)

1. Enter `try` and execute statements top-to-bottom.
2. If no exception: skip all `catch` blocks.
3. If an exception occurs: jump to the **first matching** `catch`.
4. After `try` or `catch`: always execute `finally` (cleanup).
5. Continue with the next statement after the block (or propagate if not handled).

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Example: custom checked exception

```
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String msg) { super(msg); }
}

class BankAccount {
    private double balance;
    BankAccount(double opening) { balance = opening; }
    void withdraw(double amount) throws
        InsufficientFundsException {
        if (amount > balance) throw new
            InsufficientFundsException("need " + amount);
        balance -= amount;
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Handling it

```
public class Demo {
    public static void main(String[] args) {
        BankAccount a = new BankAccount(100);
        try {
            a.withdraw(150);
        } catch (InsufficientFundsException e) {
            System.out.println("Withdraw failed: " + e.
                getMessage());
        }
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Give one checked and one unchecked exception example.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Give one checked and one unchecked exception example.
2. When does `finally` run?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Give one checked and one unchecked exception example.
2. When does `finally` run?
3. Difference between `throw` and `throws`?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Using exceptions for normal control flow.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Using exceptions for normal control flow.
- Swallowing an exception silently.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Watch-outs

- Using exceptions for normal control flow.
- Swallowing an exception silently.
- Showing raw technical exception details to end users.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Exceptions enable safe error handling without crashing.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Exceptions enable safe error handling without crashing.
- Catch specific exceptions; use `finally` for cleanup.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Key Takeaways

- Exceptions enable safe error handling without crashing.
- Catch specific exceptions; use `finally` for cleanup.
- Custom exceptions make domain errors clear.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. Why catch specific exceptions (not `Exception`)?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. Why catch specific exceptions (not `Exception`)?
2. What does "checked exception" mean?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Checkpoint Questions

1. Why catch specific exceptions (not `Exception`)?
2. What does "checked exception" mean?
3. When do we use `throws` in a method signature?

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

Next Lecture Preview

Next: Multithreading + Synchronization

- We will create threads using `Thread` and `Runnable` and see race conditions.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).